

# Inside Kahuna’s Distributed Transactions: Durable Intents, Validation, and Recovery in a Partitioned Key–Value Store

Andres Gutierrez

August 2026

**Abstract** Kahuna is a partitioned key–value store. Its transaction layer combines several mechanisms: in-memory state owned by actors, metadata replicated with Raft, hybrid logical clocks, multiversion reads, both lock-based and optimistic concurrency control, and asynchronous materialization after commit. This paper reconstructs the transaction protocol of Kahuna 1.5.3 and explains the guarantees that emerge when these subsystems interact. At the center of the protocol is a durable commit core: a canonical transaction record on an anchor partition, plus a prepared intent stored beside every modified key. The prepare step records the exact value, the commit timestamp, the participant manifest, the expiration, and, for read-before-write operations, the validated base revision. A transaction commits only after every prepare is durable and after both conflict and read-set validation pass. While committed intents wait for materialization, readers consult the canonical decision. Recovery can therefore complete an acknowledged transaction without the original coordinator. The design also includes an atomic single-partition fast path, heterogeneous Raft write coalescing, adaptive decision deadlines, idempotent operation admission, snapshot-retention leases, and barriers for range reconfiguration. We analyze which anomalies these mechanisms prevent, where the guarantees depend on transaction options, and where they deliberately stop. In particular, optimistic predicate scans do not provide full phantom protection, transactions that mix persistent and ephemeral keys are not crash-atomic, and the replica-side stale-base veto is still under fault-soak validation. The result is best described not as one universal isolation level, but as a family of execution policies built over a durable-intent commit core.

## 1 Introduction

A distributed key–value transaction looks simple from the outside. Behind its small interface, a single logical operation may touch several key ranges, be forwarded to different Raft leaders, pass through several actor mailboxes, outlive the request that started it, race a range split, or be retried after the client lost the reply. The transaction layer must make all of these mechanisms agree on one outcome, and it must keep the common case cheap.

Kahuna answers with a layered protocol. An in-memory coordinator owns the live transaction and its operation history. Key actors stage tentative multiversion (MVCC) values and hold point locks. For persistent keys, commit turns these transient effects into replicated *prepared intents*: durable records of the exact write that each participant will apply. A canonical transaction record, stored on an *anchor partition*, chooses commit or abort exactly once. The committed values can be written into the ordinary key–value log after the client already received the decision, because reads and writes know how to handle unresolved intents. Background recovery, barriers around range reconfiguration, and idempotent replay complete the protocol after failures.

This paper makes four contributions:

1. It decomposes Kahuna’s transaction path at the implementation level: routing, actor ownership, admission control, MVCC, locking, validation, durable decisions, recovery, and persistence.
2. It derives a guarantee matrix organized by anomaly, instead of attaching one unqualified isolation label to every configuration.
3. It explains the latency and throughput techniques that keep the protocol’s safety boundaries intact: an atomic single-partition fast path, partition-local heterogeneous batching, deferred settlement, consensus work performed off the actor mailbox, and bounded priority admission.
4. It identifies notable engineering combinations and their limits, including prepared-base fencing, participant-side abort assistance, settlement barriers for range moves, and a replica-side stale-base veto whose end-to-end fault campaign is still in progress.

This is a design and implementation paper, not a performance evaluation. When we say that a path needs fewer consensus barriers or fewer allocations, the claim follows from the code path. It is not a substitute for workload measurements. The analyzed releases are Kahuna 1.5.3, Kommander 1.4.4, Nixie 1.3.0, and CamusDB 0.11.1. The source code and the repository documentation are the artifact of record [1, 2, 4, 12].

## 2 System Model and Scope

### 2.1 Storage, processes, and failure assumptions

Kahuna partitions the key space. A routing layer maps a key or a key range to a logical partition, and then to the current Raft leader of that partition. Kommander provides the Raft machinery: replicated logs, leadership, quorum durability, replay, and snapshot installation [2, 3]. Nixie provides actors with bounded mailboxes [4]. A per-key actor serializes the state transitions of its key. A partition scheduler that runs outside the actor mailbox submits replicated writes, so a slow quorum does not block unrelated actor messages.

The protocol assumes crash failures, message loss or delay, retries, leader changes, and range splits or merges. It does not try to make progress when the required partition has no available quorum. Hybrid logical clock (HLC) timestamps provide a compact time domain that respects causality [5]. They do not provide the bounded clock uncertainty or the external consistency of Spanner [6]. Local monotonic clocks measure elapsed time and drive admission aging. Distributed ordering and durable deadlines are always expressed as HLC values.

Keys belong to one of two durability classes:

- **Persistent keys** are written to Raft and later flushed to the configured backend (memory, SQLite, or RocksDB). The Raft log and its write-ahead log (WAL), not the backend flush, are the immediate durability authority.
- **Ephemeral keys** live only in memory. Their transactional staging and their two-phase coordination never become durable across a crash.

A transaction configured for durable decisions rejects any modified ephemeral key. A best-effort transaction may mix the two classes, but only its persistent subset is recoverable. A crash between persistent finalization and ephemeral application can expose a partial outcome across the two classes. The API therefore makes the durability boundary visible instead of implying an atomicity that the storage substrate cannot provide.

### 2.2 Two transaction interfaces

Kahuna exposes two transaction shapes.

**Script transactions** execute a parsed sequence of statements on the server. A single statement can take the direct, non-transactional path. A multi-statement or explicit transaction goes through the coordinator.

**Interactive transactions** expose begin, read, write, scan, commit, and rollback calls over gRPC or the embedded API. The server, not the client, owns the authoritative working set. A transaction handle contains the transaction identifier, the coordinator key, and, after the first confirmed persistent

mutation, the key of the durable record anchor. REST exposes script execution and individual key operations, but not the full interactive session lifecycle.

The distinction matters when a network outcome is ambiguous. Kahuna never accepts a client-provided list of writes as a substitute for the server's staged state. When the client reuses the same handle and the same operation identity, the server can answer from the live session, from a bounded cache of terminal results, or from the canonical durable record.

## 3 Architecture

### 3.1 Data plane

The normal path has four ownership layers:

```
client request
  |
  session coordinator and operation registry
    |
    range/partition router -> current Raft leader
      |
      per-key actor: MVCC state, lock, staged value
        |
        partition batch scheduler -> Kommander Raft
```

The coordinator owns the facts that span the whole transaction: timeout, priority lease, read policy, locking policy, participant set, range locks, read observations, operation results, and finalization state. Key actors own the facts that are local to one key: committed revisions, the transaction's staged revision, point locks, write intents, and prepared-intent overlays. This split keeps a shared process-wide transaction object out of the hot key path, while it keeps one authority for commit.

Every route carries a *range generation*, a counter that changes whenever the ownership of a range changes. A write is fenced against this generation twice: when it first reaches the owner of its key, and again immediately before a queued replication batch is flushed. The second check is essential. A proposal can wait in a batch long enough for a split, a merge, or a leader move to invalidate its original route.

### 3.2 Control plane

Several replicated control structures support the data plane:

- a range map and a generation fence determine ownership;
- snapshot holds publish an MVCC retention floor;
- transaction records store canonical decisions;
- prepared-intent records store participant effects;
- completion evidence and transaction metadata survive cache eviction, replay, and range transfer;
- application-durability floors stop Raft log compaction from running ahead of asynchronous backend persistence.

These structures are more than bookkeeping. They turn local optimizations, such as caches, asynchronous materialization,

and actor-local staging, into state machines that can be recovered.

### 3.3 Admission and overload control

Before a script or an interactive session begins, it passes through a per-node priority gate. The script gate and the session gate are separate, so one workload shape cannot consume the whole allowance of the other. Configurable limits on maximum concurrency, queue depth, and admission wait bound the resource use. Reserved slots protect high and critical priorities. Monotonic aging helps queued requests make progress, although the original priority still controls access to the reserved capacity.

Admission sheds load; it does not enforce correctness. It neither preempts an active transaction nor changes the lock conflict rules. An `AdmissionRefused` response means that no transaction began, and the caller may back off. Session lifetime is clamped independently. The defaults are a 5 s transaction timeout, a 300 s maximum, a 5 s admission wait, and a 30 s maximum wait. Priority admission is configurable and can be disabled.

The replicated write scheduler adds a second overload boundary. It bounds the queued items and bytes per partition and globally. Terminal work, meaning decisions, materialization, and settlement, has reserved capacity. A flood of new writes therefore cannot indefinitely stop already-decided transactions from releasing their intents.

## 4 Live Transaction Execution

### 4.1 Begin and operation idempotency

Starting a transaction validates the combination of options, acquires admission, creates a transaction identifier derived from the HLC, and installs a session context. A fixed nonzero read timestamp cannot be combined with tracked latest-read validation, because the two express different semantics: a historical as-of view versus a dependency set over current versions.

Every interactive operation carries a 128-bit operation identifier and a digest of its request. `BeginOperation` checks the operation table and the finalization fence in one atomic step. The result is one of three cases:

- **new:** the operation may execute;
- **pending:** another copy is already executing, so the caller joins its result;
- **completed:** the saved response is returned, and the effect is not applied again.

Reuse of an identifier with a different digest is rejected. A separate cache on the participant side handles the case where a key actor applied an effect, but the coordinator did not record the completion before the reply was lost. Request admission

and effect application are therefore both idempotent, each at its own ownership layer.

Commit, rollback, and timeout reaping compete for a single finalization slot. The winner closes the session to new operations, waits for the operations admitted earlier to drain, freezes the transaction snapshot, and performs finalization. Concurrent finalizers observe the same result. This order prevents a late write from entering after the participant manifest is frozen.

### 4.2 Staging and read-your-writes

A transactional write to a persistent key does not immediately append an ordinary key-value entry to Raft. Instead, the key actor checks the write's conditions, acquires an exclusive point lock, and stores an MVCC entry tied to the transaction, together with a leased in-memory write intent. Later reads by the same transaction select this staged version. The transaction sees its own writes, and other transactions never see uncommitted data.

Client writes take the exclusive point lock in both locking modes. The modes differ mainly in how reads are protected:

- **Pessimistic reads** acquire exclusive point locks. Prefix scans acquire an exclusive prefix lock. Range scans acquire a range lock, which is exclusive by default.
- **Optimistic reads** take no read locks. They record observations for validation at commit. Optimistic locking implies read-set validation even when the explicit validation option is `None`.

The public point lock is currently exclusive rather than shared. Shared per-key read locks are a planned extension, not an implemented guarantee.

### 4.3 MVCC reads and snapshot holds

Without a fixed timestamp, a read selects the latest visible committed revision, or the transaction's own staged value if one exists. With a fixed read timestamp  $t_r$ , point reads and scans select the newest version at or before  $t_r$ . This yields a historical as-of view that is consistent across partitions for the whole transaction, as long as the required history is still retained.

A client that needs a long-lived historical view acquires a replicated, leased *snapshot hold*. If the live holds have timestamps  $h_1, \dots, h_n$ , the effective retention floor is

$$F = \min_i h_i.$$

Pruning keeps the newest revision at or below  $F$  and every newer revision. Actors keep a bounded recent history in memory: 16 revisions by default, plus the revision at the floor boundary. Deeper reads fall back asynchronously to the materialized backend, so they do not block an actor mailbox. Holds are reference-counted and leased, and they are reaped after they expire.

This mechanism protects the availability of history. It does not serialize a historical read with a later write. A transaction that reads an old snapshot and then writes does not automatically receive optimistic validation against current versions. Applications should treat fixed-timestamp transactions as read-only, unless they enforce their own invariant at the application level.

## 5 Conflict Detection and Validation

Kahuna deliberately separates four questions that are often mixed together:

1. who may stage a value for a key;
2. whether an observed version is still current;
3. whether a predicate is protected; and
4. whether a prepared write is based on the expected predecessor.

### 5.1 Point locks and dirty writes

An exclusive point lock and a write intent allow only one live transaction to stage a write for a key. This prevents dirty writes. Conflicting writers wait or retry instead of silently overwriting each other's tentative state. The lock is leased, and it is released on commit, on abort, and on the reaping paths. Locks are local to the actor, and therefore fast, but their liveness depends on renewal and on leadership.

Locking alone is not enough. A transaction can read a key before it takes the write lock, another transaction can commit in between, and the first transaction can acquire the lock afterwards. Kahuna therefore remembers the base state of every read-before-write key and validates it again during durable preparation.

### 5.2 Read-set validation

Tracked reads record (key, exists, revision). If one transaction observes the same key with inconsistent base revisions, validation fails immediately. Keys that also appear in the write set are excluded from ordinary read-set validation, because the stronger staged-base fence covers them at prepare time.

At commit, the coordinator batches probes of the current state across nodes and compares the existence and the revision of each observed key. A changed or deleted key aborts the transaction. Read-only transactions go through this path as well. Skipping validation for an empty write set would admit classic read skew.

Validation runs after all durable prepares and before the commit decision. A failed validation therefore leaves behind durable intents that are safe to abort and to recover. It never exposes tentative writes.

### 5.3 Concurrent-writer and range-lock probe

Before read validation, the coordinator issues one batched conflict probe. It asks for:

1. undecided foreign writers on the keys that this transaction only read; and
2. foreign range or prefix locks that cover the keys that this transaction writes.

Probing active writers first avoids a situation where two transactions each wait for the other's read validation. A missing probe response fails closed. The post-prepare range-lock check matters because a scan can acquire a predicate lock after a writer stages its value but before that writer decides.

Range locks are leases that live in the memory of the leader. The coordinator renews them, and it reacquires them after a participant leader change when the live session can continue. If the coordinator itself is lost, its session and its lock list disappear, and the leases eventually expire. The durable validation and decision machinery remains the backstop. Range locks should not be mistaken for a globally replicated predicate-lock manager.

### 5.4 Staged-base fencing

For every non-blind persistent write, the prepared intent records whether the base existed and which revision it had. Kahuna checks this dependency in three layers:

1. **Pre-proposal check.** The leader probes the current key state before it proposes the prepare. A foreign undecided intent produces a retry rather than a guess.
2. **Apply-time fence.** The prepared-intent state machine on every replica compares the validated base with the last transactionally committed head that it remembers. A competing commit in the window between probe and proposal makes the prepare stale. The intent is still installed deterministically, but the leader withholds a successful prepare acknowledgement, and the coordinator aborts.
3. **Replica stale-base veto.** If a replica detects a stale base while it applies a prepare, it can asynchronously drive a best-effort abort at the canonical anchor. The goal is to stop a stale leader from acknowledging a prepare that a fresher replica can prove invalid.

Blind writes carry an explicit sentinel instead of a base revision, and they keep last-writer-wins semantics. The third layer is recent hardening, motivated by kill and failover campaigns. Unit and focused regression suites cover it, but the final multi-node fault soak has not yet confirmed closure. Section 12 treats it as a promising defense, not as settled experimental evidence.

### 5.5 What anomalies are prevented?

The answer depends on the policy that the transaction uses.

| Data anomaly              | Main defense                               | Boundary                                  |
|---------------------------|--------------------------------------------|-------------------------------------------|
| Dirty read                | staged values hidden; decision-aware reads | own writes re-visible to self             |
| Dirty write               | exclusive point lock and intent            | leased, leader-local live state           |
| Lost update               | base revision at prepare/apply             | blind writes are last-writer-wins         |
| Non-repeatable point read | pessimistic lock or optimistic validation  | no validation in permissive mode          |
| Read skew                 | frozen read set and commit validation      | only observed keys participate            |
| Write skew                | read-set validation plus writer probe      | dependencies must be read/tracked         |
| Distributed anomaly       | Main defense                               | Boundary                                  |
| Predicate phantom         | prefix/range lock                          | optimistic scans track returned keys only |
| Partial durable commit    | anchor decision plus durable intents       | persistent-only durable mode              |
| Duplicate operation       | operation ID, digest, replay caches        | caller must reuse identity                |
| Stale route write         | generation fence at receipt and flush      | retry after re-resolution                 |

A pessimistic transaction that commits over correctly locked predicates protects both point reads and phantoms. An optimistic transaction that commits provides serializable-like behavior over the point dependencies it observed, including read-only transactions and read-before-write keys. However, an optimistic scan records the keys it returned, not the gaps it searched. A new key inserted into such a gap is therefore a possible phantom. It would be incorrect to describe every Kahuna transaction as serializable or as snapshot isolated. Kahuna instead exposes a policy matrix. Its guarantees are the conjunction of the read timestamp, the locking mode, the validation mode, the durability class, and the dependencies that the application declared.

## 6 The Durable-Intent Commit Protocol

### 6.1 Frozen commit input

When persistent finalization starts, the coordinator builds an immutable commit input. It contains:

- the transaction identifier and epoch;
- the coordinator key, the anchor key, the anchor partition, and the route generation;
- one canonical HLC commit timestamp;
- an absolute decision deadline;
- a hash and an explicit list of the participant keys;
- per-partition prepared intents, each with the exact final value or tombstone, the target revision, the expiration, and the validated base state.

The participant manifest prevents recovery from committing a transaction for which only an accidental subset was prepared. The identity of the input is the tuple (transaction, epoch, manifest hash). An exact replay is idempotent. A replay that diverges under the same identity is rejected.

The anchor is chosen from the first confirmed persistent mutation. Because the anchor is a key, normal routing and range migration can locate the record. No separate global transaction service is needed.

### 6.2 Replicated state machines

The anchor transaction record follows this monotonic state machine:

```
absent --initialize--> undecided --commit-->
committed
\
\--abort----->
aborted
\-----abort tombstone-----> aborted
```

Commit cannot create a record from nothing. Abort may create a tombstone, which fences a delayed initialize or commit so that it cannot resurrect an expired transaction. An undecided record accepts exactly one terminal decision. An exact replay of a terminal decision is a no-op. An identity mismatch fails.

Each participant key has at most one live prepared intent:

```
absent -> pending -> committed -> removed
\ -> aborted -> removed
```

The transition is applied deterministically on leaders, on followers, during WAL replay, and during range transfer. An exact duplicate prepare is idempotent. A different effect under the same identity, or an intent that belongs to another transaction, is rejected. Removal is allowed only after resolution has made the effect durable or has safely discarded it.

### 6.3 Standard two-phase path

For a persistent transaction that spans arbitrary partitions, finalization proceeds in seven steps.

1. **Revalidate staged bases.** Detect obvious lost updates before any consensus work is spent.
2. **Freeze serialization.** Serialize each exact prepare delta once, so that retries cannot accidentally change its bytes or its meaning.
3. **Initialize and prepare.** On the anchor partition, the initialization of the transaction record and the prepare of the anchor key share one atomic Raft proposal. The other participant partitions prepare concurrently. Completion distinguishes “batch committed” from “this prepare acknowledged,” because the deterministic apply-time fence may commit the proposal while it refuses the logical success of the prepare.
4. **Help blocked predecessors.** If a key is occupied by an intent that is decided but not yet settled, consult its canonical record, help materialize or clear it, and retry the idempotent prepare within a bound.
5. **Validate.** After every prepare is durable on a quorum, run the foreign-writer and range-lock probe, and then run read-set validation.
6. **Decide.** Propose commit or abort at the anchor. A routed read of the canonical record determines the real winner of the compare-and-set. The sender’s local projection is not treated as truth across a leader change.
7. **Resolve.** Materialize committed effects or clear aborted ones, release locks, and settle the intent records. Deferred settlement is enabled by default, so this step may continue after the durable decision is returned to the client.

This order produces a useful invariant:

A visible durable commit implies that the canonical record is committed and that every persistent participant effect already exists as a prepared intent that is durable on a quorum.

It does not imply that every effect is already rewritten as an ordinary key–value entry in the backend.

## 6.4 Adaptive presumed-abort deadline

An undecided two-phase commit must not block a key forever when the coordinator disappears. Kahuna observes recent finalization latency with a local monotonic timer and derives a window

$$W = \text{clamp}(m \cdot p_{99}, W_{\min}, W_{\max}),$$

where the default multiplier  $m$  is 4 and the default bounds are 5 s and 60 s. The absolute deadline is the commit HLC plus  $W$ . When a commit is attempted, a fresh HLC attempt timestamp is compared with the frozen deadline. An attempt beyond the deadline cannot commit.

After the deadline, a participant leader may drive an abort tombstone at the anchor. The tombstone makes presumed abort durable and prevents a delayed coordinator from

choosing commit later. The observation window adapts to the latency of the deployment, while the safety comparison lives in the replicated HLC domain and does not depend on synchronized wall clocks.

This is a trade-off between liveness and safety. A very short window increases aborts during transient slowness. A long window keeps orphaned intents alive longer. Neither choice removes the need for an anchor quorum.

## 6.5 Single-partition fast path

If all persistent participants and the anchor sit on one partition that the local node leads, Kahuna can place the record initialization, all prepares, and the commit decision in one atomic Raft proposal. A preflight step checks for foreign intents and validates reads. A late staged-base check narrows the last race before submission. After the proposal commits, the leader installs the committed intent view before it replies.

In a multi-process Raft group, the fast path is disqualified in two cases: when the transaction has a read dependency beyond the keys it writes, or when a write carries a validated base whose safety would otherwise rest on a check performed before a proposal that might stall. This conservative rule prevents a queued atomic bundle from applying after a conflicting update that happened after validation. A single-process Raft group can safely relax some of these constraints, because it has no remote replica divergence and no inter-node proposal delay.

The fast path is therefore not simply “one partition means one phase.” Eligibility depends on the dependency footprint and on the execution model. When the coordinator cannot prove safety, it falls back to the standard protocol.

## 7 Decision-Aware Visibility and Settlement

### 7.1 Why commit can precede materialization

Synchronous materialization would add another replicated write per participant to the client’s latency. Kahuna instead separates the *logical decision* from the *physical representation* of committed data.

Suppose key  $k$  has an old committed value  $v_0$  and a prepared intent that contains  $v_1$  at commit timestamp  $t_c$ .

- A snapshot read at  $t < t_c$  returns  $v_0$  without consulting the decision.
- A latest read, or a snapshot read at  $t \geq t_c$ , consults the anchor.
- If the record is committed, the read overlays  $v_1$ , even before materialization.
- If the record is aborted, the read returns  $v_0$ .
- If the record is undecided or temporarily unreachable, the read waits or retries rather than exposing a guess.

Scans apply the same overlay across all intents. A later writer must first resolve a committed predecessor before it derives a new revision or evaluates its conditions. It cannot build on stale materialized state.

## 7.2 Settlement sequence

For a commit, a resolver writes the intent's value as an ordinary transactional key-value log entry, waits until the leader applies it, and only then atomically marks or removes the prepared intent. If any step fails, the intent remains, and recovery can retry. For an abort, the resolver clears the staged state and settles the intent without materializing a value. All of these operations are identity-checked and idempotent.

This order handles a crash at each dangerous point: after the decision but before materialization, after materialization but before the acknowledgement, and after the acknowledgement but before the intent is removed. In every case, the durable record chooses the outcome, and at least one durable representation of the effect remains available.

Completion evidence lets a participant recognize an operation that is already committed after its actor state or its recent-result cache is gone. The evidence is included in persistence and in range transfer, so a key cannot lose its transaction identity just because it moved.

## 7.3 Recovery

Recovery sweeps the prepared intents on the current leader of each participant partition:

- Committed record: materialize and settle.
- Aborted record: clear and settle.
- Undecided record before the deadline: leave the intent pending.
- Undecided record after the deadline: attempt a canonical abort.
- Missing record after the permitted window: create an abort tombstone.

Every action rechecks the result of the canonical compare-and-set. A recovery worker that asks for abort but loses the race to a valid commit must follow the committed decision. Once preparation and the anchor record are durable, the original coordinator is no longer needed.

# 8 Reconfiguration, Leadership, and Persistence

## 8.1 Leader changes

Actor caches and live locks belong to one node. They are not automatically coherent across replicas. Correctness after a promotion comes from replaying the deterministic transaction-record and prepared-intent state, and then consulting those structures instead of trusting a cache. Requests are

routed to the current leaders, and reads of the canonical record may be re-routed when leadership changes.

The difficult case is a promoted replica that holds an orphaned prepared intent and remembers an older per-key committed head than another replica observed. The apply-time base fence helps only if the leader that evaluates the logical acknowledgement has the necessary history. The replica stale-base veto extends detection to every replica that applies the prepare and asks it to force an anchor abort. This defense targets the gap directly, but its final partition and kill soak remains an open validation item.

Live range locks and point locks are not reconstructed as durable lock ownership. They expire, or a surviving coordinator reacquires them. Safety after a coordinator loss rests instead on durable prepared intents, base validation, the canonical decision, and recovery.

## 8.2 Range splits and merges

An intent cannot move like an ordinary value unless the destination also keeps everything needed to resolve it. Kahuna therefore enforces a settle-before-cutover barrier:

1. scan the moving interval for prepared intents;
2. resolve every decided intent;
3. refuse the transfer while an undecided intent remains or while the scan cannot be trusted;
4. transfer the values, the completion evidence, and the remaining metadata;
5. change the routing generation.

Materialization itself re-resolves the key's current partition rather than assuming the partition captured at prepare time. These two measures address opposite races. The barrier prevents a move from dropping an overlay. Re-resolution prevents a late resolver from writing to the old owner.

Every queued write also carries its range generation and is rechecked at batch flush. A stale operation is released for a routing retry without contaminating the valid siblings in the same batch.

## 8.3 Raft versus materialized storage

Kommander's replicated WAL establishes the immediate durable order. A background writer later persists committed key revisions and transaction metadata to the selected backend. Reads can use resident replicated state before that flush. An application-durability floor prevents Raft compaction from discarding log entries that the backend has not yet made independently recoverable.

This arrangement separates consensus latency from backend write amplification, but it also creates an invariant: snapshot and compaction policy must advance only behind materialization. It would be incorrect to treat the backend as current merely because the client saw the commit.

## 9 Performance-Oriented Design

### 9.1 Heterogeneous partition batching

Kahuna’s partition scheduler accepts typed submissions: ordinary key writes, transaction records, and prepared intents. It coalesces independent operations headed for the same partition into one `Kommander ReplicateEntries` call. A default linger of 1 ms, measured from the oldest item, provides a small batching window. Batches are capped at 512 items and 4 MiB. An oversized but valid item runs alone. Only one batch is in flight per partition, while different partitions progress concurrently.

Unlike transaction-level batching, heterogeneous coalescing does not require unrelated clients to share a transaction. Each item keeps its own result and its own range fence, while the shared proposal amortizes the cost of serialization, network, quorum, and WAL. A client that cancels stops waiting for its item, but it does not cancel a consensus batch shared with other clients. Accepted items complete exactly once at the scheduler boundary.

Kommander can additionally group WAL work across partitions. The two layers exploit different locality: Kahuna groups logical entries per Raft partition, and the WAL layer may group physical persistence across partitions.

### 9.2 Consensus-barrier reduction

On the standard path, the anchor initialization and the anchor prepare share one proposal, and the non-anchor prepares run concurrently. The decision requires a second anchor proposal. On an eligible single-partition transaction, initialization, preparation, and decision share one proposal. In both cases, deferred settlement removes materialization from the client-visible critical path.

Counting barriers is more informative here than claiming a universal latency number. Network topology, quorum placement, batch density, backend choice, contention, and read validation all affect measured latency. The structural result is this: the common local case needs one consensus decision barrier, while a multi-partition commit needs prepare durability plus a terminal decision barrier.

### 9.3 Actor and allocation discipline

Per-key actors make key-local state mutation sequential and keep critical sections explicit. Consensus I/O is submitted off the mailbox, so unrelated reads or lock-release messages do not wait for a network round trip. Participant work is grouped by partition. Validation uses batched probes rather than one remote request per key. Prepare deltas are serialized once, before the retry loops.

These choices remove obvious network patterns that scale with the number of participants, and they avoid repeated serialization. Bounded mailboxes, byte caps on queues, trans-

action timeouts, leases on participant effects, and terminal reservations bound the retained state under overload. They also make overload visible as backpressure or retry instead of unbounded allocation.

### 9.4 Costs of the design

The optimizations do not make transactions free. A persistent transaction creates an anchor record and one prepared intent per written key, performs validation probes, and later materializes and removes each intent. Deferred settlement moves work out of the foreground latency, but it can lengthen the interval during which readers consult transaction records and writers help their predecessors. Heterogeneous batching trades up to the linger interval for density. Pessimistic scans can restrict concurrency over large ranges. Snapshot holds increase revision retention and backend space.

The system exposes metrics for queueing, batch fill, settlement failure, recovery, WAL phases, and related paths. These trade-offs can therefore be measured rather than inferred from throughput alone.

## 10 API Semantics Under Ambiguity

Distributed clients must interpret outcomes conservatively:

- **Committed:** the canonical persistent decision is commit, and the participant effects were prepared before that decision. Materialization may still be running.
- **Aborted:** a canonical terminal abort exists. Causes include a conflict, an explicit rollback, a validation failure, or presumed abort. The caller must start a new transaction for new work.
- **MustRetry:** no safe terminal answer was established at this boundary. The caller should retry with the same transaction handle and, for an operation, the same operation identifier.
- **AdmissionRefused:** no transaction started. The caller should back off or lower the load.

A timeout or a lost connection is not proof of abort. Starting an unrelated replacement transaction can duplicate application effects if the original later turns out to be committed. Durable handles, operation digests, retention of terminal results, and lookup of the canonical anchor are designed to make retry under the same identity the natural safe behavior.

Direct non-transactional writes have a different retry contract. If the transport reports an indeterminate outcome and the operation has no application-level idempotency key, a blind replay can repeat a condition or a mutation. The stronger operation registry belongs to interactive transactions.

## 11 Relationship to Prior Systems

Kahuna’s foundation is conventional in the best sense. Raft replicates partition state [3], HLCs provide timestamps com-



patible with causality [5], and two-phase commit separates participant preparation from a terminal decision [7]. The interest lies in how these tools are composed around a data path built on key actors.

**Percolator** implements distributed transactions over a key-value substrate with timestamped locks and a distinguished primary [8]. Kahuna’s anchor record plays a similar role as the decision authority. However, Kahuna’s record is an explicit replicated state machine, its participant effects are typed Raft entries, and its live coordinator supports both locking and validation policies.

**CockroachDB** also uses transaction records, write intents, MVCC, and asynchronous intent resolution [9]. Kahuna shares the principle that a committed logical value can be read through an unresolved intent. Kahuna’s distinctive emphasis is the integration with per-key actors, heterogeneous batching of partition proposals, explicit range-generation fences, and configurable server-owned interactive sessions.

**FoundationDB** uses optimistic conflict ranges and offers strict serializability [10]. Kahuna’s optimistic point validation is conceptually similar to checking declared dependencies, but Kahuna does not currently turn every optimistic scan into a conflict range. Its guarantee is therefore intentionally more conditional.

**Spanner** combines two-phase commit, Paxos replication, MVCC, and TrueTime to provide externally consistent transactions [6]. Kahuna uses HLC without a bounded-uncertainty commit-wait protocol, and it claims no external consistency.

**Calvin** avoids distributed commit coordination during execution by ordering transactions deterministically in advance [11]. Kahuna instead supports dynamic, interactive transactions whose read sets and participant sets emerge during execution. It pays the cost of prepare and decision coordination to keep that flexibility.

We do not claim that any individual mechanism is new. The notable features are the engineering combinations:

- atomic anchor initialization plus anchor preparation, with an eligibility-checked one-proposal commit fast path;
- exact-value prepared intents that double as the read overlay during deferred settlement;
- a three-layer defense against lost updates that ends with abort assistance from participant replicas;
- adaptive HLC decision deadlines that make presumed abort a replicated state transition;
- refusal of range moves until the decision overlays are safe to transfer;
- heterogeneous batching across unrelated ordinary and transactional entries that preserves per-item fencing and completion;
- a server-owned idempotency registry joined atomically with finalization admission.

Their value is not novelty in isolation. It lies in closing the gaps between the transaction protocol and the operational realities of actors, retries, reconfiguration, and asynchronous persistence.

## 12 Limitations and Threats to Validity

First, this analysis is tied to one source snapshot and its documented tests. It is not a formal proof. The code contains deterministic state machines and focused fault tests, but an implementation can still violate a derived invariant through a path that was not modeled.

Second, the strongest isolation statement depends on configuration. Optimistic point reads are validated, but optimistic scans validate the returned keys rather than the gaps, so phantoms remain possible. Pessimistic predicate locks are leased, in-memory, and local to the leader. Fixed historical snapshots protect time-consistent reads and retention, not automatic serialization of later writes. Shared point-read locks are not yet implemented.

Third, crash atomicity covers the persistent participant set in durable mode. Ephemeral participants cannot be recovered by their nature, and a mixed best-effort transaction can become partially visible after a crash.

Fourth, the availability of two-phase commit remains bounded by consensus and by decision recovery. A partition that cannot reach a quorum cannot safely decide. Presumed abort limits indefinite blocking by orphans after a deadline. It does not create availability during quorum loss.

Fifth, the most recent per-key failover hardening needs further empirical validation. A partition and kill campaign found executions in which a promoted node combined an orphaned prepared intent with stale per-key committed-head state, so acknowledged results could fork after failover. Kahuna 1.5.3 adds a replica-side stale-base veto, and the focused tests are green, but the repository’s acceptance gate still calls for an extended multi-node fault soak. Until that gate passes, the correct claim is “designed and locally validated to close the stale-leader gap,” not “demonstrated invariant under the full campaign.”

Finally, this paper reports no fresh benchmark. The one-phase path, batching, and deferred settlement remove or amortize identifiable work. Actual throughput, tail latency, recovery time, and storage amplification require reproducible experiments across contention levels, participant counts, backends, and fault schedules.

## 13 Conclusion

Kahuna’s persistent transaction system is organized around one compact safety idea: prepare every exact participant effect durably, choose one canonical outcome, and make every read, retry, recovery worker, and range move honor that

outcome until ordinary storage catches up. The surrounding subsystems exist to protect that idea at their boundaries. Actor ownership prevents local races. Operation identities prevent duplicate effects. Locks and validation protect declared dependencies. Base fences close the race between read and prepare. Raft makes records and intents replayable. Decision-aware reads allow deferred materialization. Deadlines and tombstones resolve abandoned work. Range barriers keep reconfiguration from dropping unresolved state.

The design brings a durable, recoverable transaction core to a high-performance actor-based key-value layer without forcing every workload through the same concurrency policy. That flexibility is also why its guarantees must be stated precisely. Persistent-only durable transactions, complete dependency tracking, and predicate locking where phantoms matter can support strong executions. Permissive reads, optimistic scans, historical writes, and ephemeral participants deliberately weaken the result. The clearest description is therefore not a single isolation label, but a protocol core plus an explicit guarantee envelope.

## 14 References

- [1] Kahuna Project. *Kahuna source tree*, version 1.5.3, 2026. <https://github.com/kahunakv/kahuna>
- [2] Kahuna Project. *Kommander source tree*, version 1.4.4, 2026. <https://github.com/kahunakv/kommander>
- [3] D. Ongaro and J. Ousterhout. “In Search of an Understandable Consensus Algorithm (Extended Version).” *USENIX ATC*, 2014. <https://web.stanford.edu/~ouster/cgi-bin/papers/raft-extended.pdf>
- [4] Kahuna Project. *Nixie source tree*, version 1.3.0, 2026. <https://github.com/kahunakv/nixie>
- [5] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone. “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases.” *OPODIS*, 2014. <https://cse.buffalo.edu/~demirbas/publications/hlc.pdf>
- [6] J. C. Corbett et al. “Spanner: Google’s Globally-Distributed Database.” *OSDI*, 2012. <https://research.google/pubs/spanner-googles-globally-distributed-database-2/>
- [7] J. Gray and L. Lamport. “Consensus on Transaction Commit.” *ACM Transactions on Database Systems*, 31(1), 2006. <https://arxiv.org/abs/cs/0408036>
- [8] D. Peng and F. Dabek. “Large-scale Incremental Processing Using Distributed Transactions and Notifications.” *OSDI*, 2010. <https://research.google/pubs/large-scale-incremental-processing-using-distributed-transactions-and-notifications/>
- [9] Cockroach Labs. “CockroachDB Architecture and Design.” <https://github.com/cockroachdb/cockroach/blob/master/docs/design.md>
- [10] J. Zhou et al. “FoundationDB: A Distributed Unbundled Transactional Key Value Store.” *SIGMOD*, 2021. <https://www.foundationdb.org/files/fdb-paper.pdf>
- [11] A. Thomson et al. “Calvin: Fast Distributed Transactions for Partitioned Database Systems.” *SIGMOD*, 2012. <https://dsf.berkeley.edu/cs286/papers/calvin-sigmod2012.pdf>
- [12] Kahuna Project. *CamusDB source tree*, version 0.11.1, 2026. <https://github.com/kahunakv/camusdb>

## 15 Appendix: Implementation Map

The following source areas are the shortest path to reproduce the implementation trace in this paper:

- `Kahuna.Core/KeyValues/Transactions/TransactionCoordinator.cs`: session life-cycle, finalization, validation ordering, and retry outcome mapping.
- `Kahuna.Core/KeyValues/Transactions/Data/TransactionContext.cs`: authoritative working set, read observations, range locks, and operation registry.
- `Kahuna.Core/KeyValues/Transactions/DurableTransactionFinalizer.cs`: standard prepare/decide/resolve path and one-partition fast path.
- `Kahuna.Core/KeyValues/Transactions/DurableFinalizeInputBuilder.cs`: frozen participant manifest, commit timestamp, deadline, and base-revision encoding.
- `Kahuna.Core/KeyValues/Transactions/TransactionRecordStore.cs`: canonical decision state machine.
- `Kahuna.Core/KeyValues/Transactions/PreparedIntentStore.cs`: participant state machine, uniqueness, base fence, and replay.
- `Kahuna.Core/KeyValues/Transactions/DurableTransactionRecovery.cs`: decision lookup, presumed abort, materialization, and settlement.
- `Kahuna.Core/KeyValues/KeyValueActor.cs`: key-local MVCC, locks, transactional staging, and intent overlays.
- `Kahuna.Core/KeyValues/Writes/`: bounded heterogeneous partition batching and Raft submission.
- `Kahuna.Core/KeyValues/SnapshotHoldService.cs` and `Ranges/SnapshotFloorStore.cs`: replicated snapshot holds and the MVCC retention floor.
- `Kahuna.Core/KeyValues/LocalLockOperations.cs`, `RoutedLockOperations.cs`, and `Handlers/RangeLockChecks.cs`: point, prefix, and range lock ownership and lease behavior.
- `Kahuna.Core/KeyValues/Ranges/DataPartitionRouter.cs` and `RangeStateTransferService.cs`: routing generation and split/merge fencing.